Лабораторна робота 3

Класи об'єктів на прикладах геометричних фігур

Завдання: навчитися програмувати прості класи об'єктів на прикладах геометричних фігур.

1. Створити новий проект типу «Додаток Windows Form» - графічну форму для відображення об'єктів створюваного класу.

2. У проекті створити окремий файл, і в ньому запрограмувати клас, що описує геометричну фігуру відповідно до варіанта завдання.

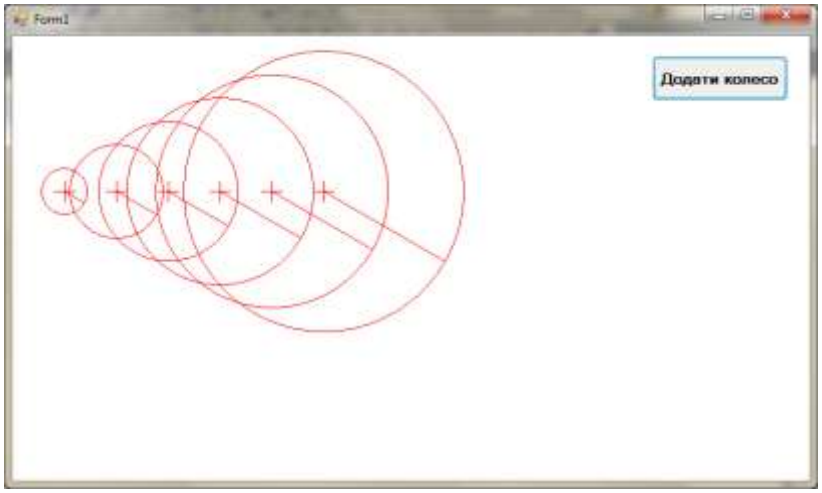
3. В клас ввести статичне поле для підрахунку кількості примірників об'єктів цього класу.

4. В клас ввести поля, що задають геометричні розміри фігури, їх колір і інші параметри.

5. Запрограмувати конструктор з параметрами для об'єктів цього класу.

6. На формі розмістити кнопку і запрограмувати її на створення чергового екземпляру об'єкта цього класу. При кожному натисканні кнопки повинен створюватися і відображатися на екрані новий екземпляр геометричної фігури. Розмір, розташування та інші параметри фігури повинні обчислюватися відповідно до підраховують автоматично порядковим номером примірника.

7. Перевірити працездатність створеної програми. Приклад роботи такої програми для об'єктів класу «Колесо зі стрілкою» показаний на наступному рисунку:



При вирішенні цієї задачі відповідно до наведеного вище завдання виконано наступне:

1. У проект включений файл **`Gear1.cs`** з описом класу **`Gear0:`**

```
// Цей файл проекту створюється повністю програмістом для опису своїх класів.
// Щоб програма запрацювала, методи цих класів повинні викликатись
// в результаті натискання кнопок миші на діалоговій формі, натискання клавіш, спрацьовування таймерів
// чи виникненні будь-яких інших подій.
// Цей файл передує файли проекту, створення яких повністю або частково автоматизовано:
// - Program.cs - містить точку входу до програми (функцію main), генерується повністю автоматично
// У момент створення проекту.
// - Form1.Designer.cs - перша половина опису класу Form1, генерується повністю автоматично
/ / При малюванні форми та розміщення на ній елементів управління.
// - Form1.cs – друга половина опису класу Form1. Автоматично генерується лише її заготівля.
// Заповнюється програмістом, щоб ввести у форму нові поля даних та реалізувати
// Функції елементів управління
using System;
using System.Collections;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Drawing;
using System.Windows.Forms;
```

```
namespace Mechan_01
{

 public class Gear0 // Клас нерухомих коліс.
    { // Параметри, загальні всім екземплярів коліс:
        public static int GearCount = 0; // Лічильник для присвоєння кожному колесу унікального номера.
                                         // Параметри та властивості окремого колеса:
        Point center; // Центр колеса(координати в пікселях).
            int radius; // Радіус колеса (розмір у пікселях).
        Form1 form; // Посилання на форму, на якій відображається колесо.
        double angle = 30; // поточний кут повороту колеса
        //(у градусах, відраховується за годину. стрілці від осі X.)
        public double Angle { get { return angle; } }
       // Властивість колеса дозволяє дізнатися (але не змінити) його поворот.

       // КОНСТРУКТОРИ:
        public Gear0(Point c, // Точка центру колеса
                     double m // Модуль зуба для розрахунку радіуса колеса
                        int n, // Число зубів
                   Form1 form1 // Форма для відображення
                     )
        {// Оскільки створюється нове колесо, збільшимо значення лічильника коліс на 1:
            ++GearCount;
            // Заповнимо поля об'єкта відповідно до заданого модуля і числа зубів,
            center = c;
            radius = form1.scale * (int)(m * n) / 2;
            // Запам'ятаймо посилання на форму, на якій розташоване це колесо:
            form = form1;
            // Після завершення конструювання колесо малюється на формі.
            Draw(form.pen1, form.graph);
        }
        // МЕТОДИ:
        public void Draw(Pen pen, Graphics graph)
        {// Метод відображає колесо на поверхні graph кольором pen.
         // Малюємо перехрестя в центрі колеса:
            graph.DrawLine(pen, center.X - 10, center.Y, center.X + 10, center.Y);
            graph.DrawLine(pen, center.X, center.Y - 10, center.X, center.Y + 10);
            // Малюємо коло з покажчиком кута повороту колеса:
            graph.DrawPie(pen, center.X - radius, center.Y - radius,
                2 * radius, 2 * radius, (int)Angle, 360);
        }
        public void Show() { Draw(form.pen1, form.graph); } // Відображення, це малювання пером №1
        public void Hide() { Draw(form.pen2, form.graph); } // Стирання, це малювання пером №2
    }
}
```

2.

Розглянемо структуру цього файлу докладніше:

Для підрахунку кількості коліс на рисунку в клас **Gear0** введено статичне поле `public static int GearCount = 0; // Лічильник для присвоєння кожному з // коліс унікального номера.`Для опису зображення колеса в клас **Gear0** введені поля:

```
Point center; // Центр колеса(координати в пікселях).
int radius; // Радіус колеса (розмір у пікселях).
double angle = 30; // поточний кут повороту колеса (у градусах,
                   // відлічується за годину. стрілці від осі X.)
```

Щоб в об'єкті було відомо, на якій формі відображається деталь, вводимо в клас посилання на цю форму. Тип **Form1** взятий з файлу, який автоматично створений конструктором форм:

```
Form1 form; // Посилання на форму, на якій відображається колесо.
```

Запрограмований конструктор з параметрами для колеса:

```
    public Gear0(Point c, // Точка центру колеса
                 double m // Модуль зуба для розрахунку радіуса колеса
                    int n, // Число зубів
                Form1 form1 // Форма для відображення
                  )
    {
        ++GearCount;
        center = c;
        radius = form1.scale * (int)(m * n) / 2;
        form = form1;
        Draw(form.pen1, form.graph);
    }
```

Зауважимо, що модуль і число зубів хоч і передаються в конструктор, в самому об'єкті поки не зберігаються, тому що вони не використовуються явно при малюванні колеса.

3. На формі розташована кнопка button1 і для неї запрограмована функція **button1_Click**. Текст цієї функції знаходиться в файлі **Form1.cs**:

```
        private void button1_Click(object sender, EventArgs e)
        { // Обробник натискання кнопки. Створює нове колесо

            Point p1 = new Point(scale * (Gear0.GearCount * 10 + 10), scale * 30);
            // Центр колеса на формі (переведено з мм на пікселі)
            g0 = new Gear0(p1, 1.5, (Gear0.GearCount + 1) * 6, this);
            // Нове колесо з центром p1 цієї форми.
            // Модуль зубів – 1.5,
            // кількість зубів – на 6 більше, ніж у попереднього колеса.
        }
```

Для відображення колеса на формі виконано наступне:

а) в файлі **Form1.cs** оголошені графічні інструменти:

```
public Graphics graph;
     // поверхность диалоговой формы, на которой рисуются детали
public Pen pen1, pen2, pen3; // перья разных цветов для рисования,
                             // в т.ч. pen2 для стирания линий, имеет цвет фона.
```

б) в тому ж файлі оголошена посилальна змінна для розміщення колеса в пам'яті і масштабний коефіцієнт для його відображення на екрані:

```
    Gear0 g0; // Єдина посилальна змінна типу "колесо".
              // Служить до створення у пам'яті чергового колеса.
    public int scale = 5; // масштаб малювання:
                          //до пікселів на 1 мм реального розміру деталі
```

в) в файлі в файлі **Gear1.cs** описаний метод малювання колеса. Колір задається пером :

```
    public void Draw(Pen pen, Graphics graph)
      {   graph.DrawLine(pen, center.X - 10, center.Y,
                              center.X + 10, center.Y);
          graph.DrawLine(pen, center.X, center.Y - 10,
                              center.X, center.Y + 10);
          graph.DrawPie(pen, center.X - radius, center.Y - radius,
                        2 * radius, 2 * radius, (int)Angle, 360);
      }
```

г) в тому ж файлі описані методи відображення і стирання колеса. Вони зводяться до простого виклику методу малювання із зазначенням, яким пером це робити:

```
    public void Show() { Draw(form.pen1, form.graph); } // відображення
    public void Hide() { Draw(form.pen2, form.graph); } // Стирання
```

Варіанти завдань:

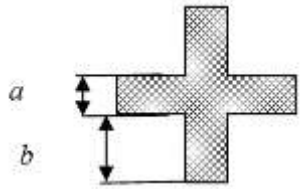
1. Геометрічна фігура «Смайлік»: його екземпляр містить розмір радіуса особи R і радіуса r очей, його конструктор без параметра створює екземпляр зі значенням 0, а конструктор з параметрами створює екземпляр з відповідним значенням R і r.



2. Геометрічна фігура «хрест», такий, що:

a) його екземпляр містить розмір два розміри a і b.

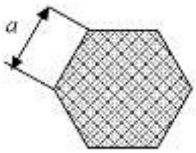
b) його конструктор без параметра створює екземпляр зі значенням 0, а конструктор з параметрами створює екземпляр з відповідним значенням a і b



3. Геометрічна фігура «Шайба» правильний шестикутник, такий, що:

a) його екземпляр містить розмір сторони a.

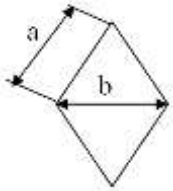
b) його конструктор без параметра створює екземпляр зі значенням 0, а конструктор з параметрами створює екземпляр з відповідним значенням a.



4. Геометрічна фігура «Ромб»:

a) його екземпляр містить розміри a і b.

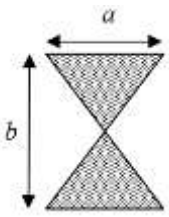
b) його конструктор без параметра створює екземпляр зі значенням 0, а конструктор з параметрами створює екземпляр з відповідним значенням a і b.



4. Геометрічна фігура "пісочний годинник" :

a) його екземпляр містить розміри a і b.

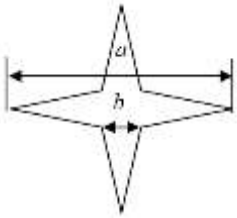
b) його конструктор без параметра створює екземпляр зі значенням 0, а конструктор з параметрами створює екземпляр з відповідним значенням a і b.



5. Геометрічна фігура «чотирикутний зірка»:

a) його екземпляр містить розміри a і b.

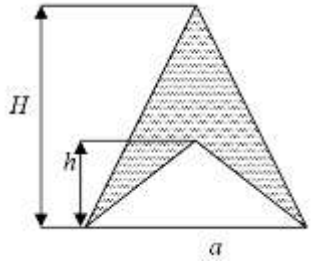
b) його конструктор без параметра створює екземпляр зі значенням 0, а конструктор з параметрами створює екземпляр з відповідним значенням a і b.



6. Геометрічна фігура «Вігвам» (Заштрихована фігура):

a) його екземпляр містить розміри a, H і h.

b) його конструктор без параметра створює екземпляр зі значенням 0, а конструктор з параметрами створює екземпляр з відповідним значенням.



7. Геометрічна фігура «еліпс»
8. Геометрічна фігура «правильний п'ятикутник»
9. Геометрічна фігура «трапеція»

a) його екземпляр містить розмір двох основ і висоти.

b) його конструктор без параметра створює екземпляр зі значенням 0, а конструктор з параметрами створює екземпляр з відповідним значенням основ і висоти.